Research on Software Reuse and Extensibility of Micro-service Architecture in the Composing Environment

Zhuwen Dong
Jiamusi University, Jiamusi, Heilongjiang 154002, China
469098894@qq.com

***Abstract*—Due to the high coupling of functional modules and the difficulty of dynamically expanding resources, the traditional single-structure composition tools can no longer meet the needs of real-time multi-person collaboration, AI-assisted composition and other complex scenes. Aiming at the special requirements of the composing environment for real-time, data consistency and artistic expression coherence, this study proposes a design scheme of micro-service architecture for composing scenes. Based on domain-driven design (DDD), the architecture decouples the system into access layer, business service layer, data layer and infrastructure layer. Through standardized service interface, containerized deployment and WebSocket+CRDT real-time synchronization protocol, high cohesion and low coupling service splitting and efficient cooperation are realized. The research further designs software reuse strategy (standardized service contract, common component library, containerized template) and extended performance optimization method (elastic scaling, asynchronous cache, database read-write separation and dynamic load balancing). The experimental results show that the P95 response delay of the system is stable at 45ms (lower than the target of 50ms), the audio rendering throughput is 3.2 times that of the single architecture, the resource utilization rate is improved by more than 60%, and the code development amount of new function development is reduced by 74% on average through service reuse. This study provides a feasible performance benchmark and design paradigm for the engineering practice of cloud-based composition tools.**

***Keywords*—Micro-service architecture, Composing environment, Software reuse, Extended performance, Real-time collaboration, Domain-driven design**

I. Introduction

With the vigorous development of digital music industry, the composing tools are undergoing a transformation from traditional local software to cloud, collaboration and intelligence [1]. Users' demand for functions has expanded from single sequence editing to complex scenes such as AI-assisted composition, real-time multi-person collaboration and seamless synchronization across platforms [2]. The functional modules of the traditional single-structure composing tool are highly coupled, and the new features need to reconstruct the whole system, and the development cycle is as long as several months; When the number of users increases sharply, it is difficult for the single architecture to dynamically allocate resources, which leads to high delay in real-time audio processing and seriously affects the creative experience.

Micro-service architecture has become the mainstream solution of distributed system because of its characteristics of "independent deployment, loose coupling and elastic expansion" [3]. Netflix achieves high availability of global streaming media services through microservices, while Spotify optimizes the response speed of music recommendation system by using service grid [4]. Micro-service architecture has obvious advantages in dealing with scenarios with high concurrency, low latency and rapid iteration of functions.

In the field of music creation, the application of micro-service architecture is still in the exploratory stage. Most of the existing research focuses on the design of general micro-services, while the requirements of composing tools for real-time, data consistency and artistic expression coherence are much higher than those of conventional software. For example, when many people collaborate to compose music, the modification of a note needs to be synchronized to the sequence view of all participants, and at the same time, the harmony analysis service is triggered to recalculate tonality. Therefore, directly applying the traditional microservice model may lead to performance crash or creative interruption.

In this study, the core goal is to improve the development efficiency and running performance of composing tools, and the microservice splitting standard and interaction protocol for composing scenes are proposed, and a distributed system model supporting real-time collaboration is constructed. Design standardized service interface and containerized deployment scheme to improve the reuse rate of general functional modules; Experiments show that the response delay of the system is less than 50ms and the resource utilization rate is 60% higher than that of the single architecture in the scenario of 10,000-level concurrent users.

II. Design of Micro-service Architecture for Composition Scenes

The architecture proposed in this study defines the system boundary through domain-driven microservices splitting, and standardized interfaces and containerization lay the foundation for software reuse. In this paper, the hierarchical micro-service architecture based on domain-driven design (DDD) is adopted to decouple the complex composition system into independent services with high cohesion and low coupling [5-6]. As shown in Figure 1, the architecture is divided into four layers.

© 2026, copyrighted by Saxion University of Applied Sciences.

DOI 10.23919/EDPEECPS00009.2026.00059
Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 09:44:08 UTC from IEEE Xplore. Restrictions apply.

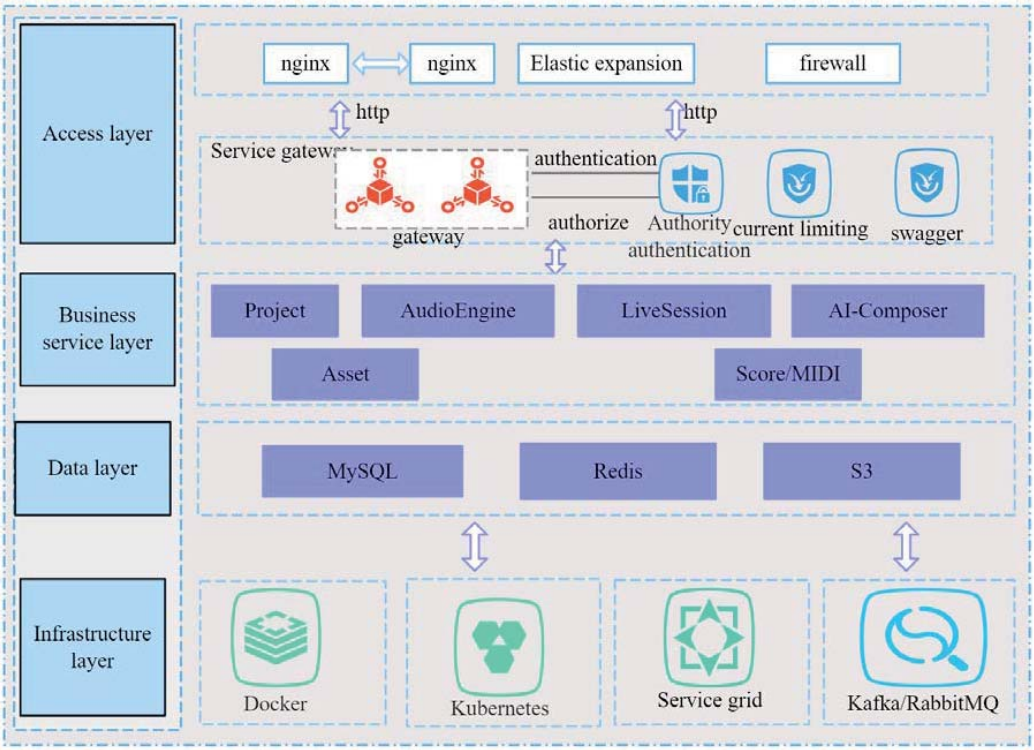


Fig. 1. Hierarchical microservice architecture based on DDD

The unified entrance of the access layer handles authentication, routing and current limiting, and distributes the protocol WebSocket for real-time collaboration and HTTP for REST to downstream services after conversion [7].

In the business service layer, the project service manages the music project metadata (title, author, timeline structure). Audio engine service is a core computing-intensive service, which is responsible for non-real-time sequence synthesis and rendering of effectors. Scalable horizontally to process multiple rendering tasks in parallel. Real-time collaboration service manages collaboration room status, broadcasts operation events and resolves conflicts based on WebSocket [8]. AI composition service encapsulates AI model and provides RPC interfaces such as harmony generation and melody continuation. Music score and MIDI service deal with music score display logic and MIDI message parsing/generation. User asset service manages static resources such as samples and audio sources uploaded by users.

The data layer selects different databases according to the data characteristics-MySQL for project metadata, Redis for real-time session state, and object storage for audio files [9].

The infrastructure layer includes container, Kubernetes, service grid (Istio for observation and governance) and message queue (Kafka/RabbitMQ).

Service split interface definition:

```
paths:
  /projects/{projectId}/collaborators:
    patch:
      summary: Add project collaborators
      requestBody:
        content:
          application/json:
            schema:
              $ref:
'#/components/schemas/AddCollaboratorRequest'
      responses:
        '202': # Asynchronous acceptance
          description: Request accepted, notifying collaboration service through message queue
```

Pseudo-code of core event handling in real-time collaboration service:

```
class LiveSessionService:
    async def handle_note_event(self, session_id: str, event: NoteEvent):
        # 1. Conflict-free merge using CRDT
        merged_state = self.crdt_merge(session_state, event)
        # 2. Persistence to Redis (low latency)
        await redis.set(f"session:{session_id}", merged_state)
        # 3. Broadcast to other users in the room through WebSocket
```


Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 09:44:08 UTC from IEEE Xplore. Restrictions apply.

await websocket_manager.broadcast(session_id, event)

The synchronous call of communication between services uses gRPC (high performance, suitable for mass data transmission between audio engine and AI service); Asynchronous event publishing/subscribing uses message queue [10]. Real-time data synchronization adopts WebSocket+CRDT (conflict-free replication data type) protocol. Each editing operation is broadcast through WebSocket as an operation transformation (OT) or CRDT increment to ensure final consistency and high response speed.

III. Optimization Method of Software Reuse and Extended Performance

A. Software Reuse Strategy

(1) Standardized service contract

All services clearly define interfaces through Protocol Buffers (gRPC) or OpenAPI (REST) [11]. For example, the audio rendering request AudioRenderRequest contains unified parameters (sampling rate, format, effect chain configuration), so that different clients (Web, desktop) can reuse the same service.

(2) Common component library

The common logic in the music field, such as pitch conversion, beat calculation and harmony analysis algorithm, is encapsulated into an independent SDK or an internal public library, and the version is managed through the private warehouse Nexus for reference by various services.

(3) Containerization and templating

As shown in Figure 2, each service and its dependencies are packaged as a Docker image [12]. Using the Helm Chart or Kustomize of Kubernetes to define the deployment template, the new service instance can be deployed with one click, which greatly improves the consistency and deployment efficiency of the development environment.

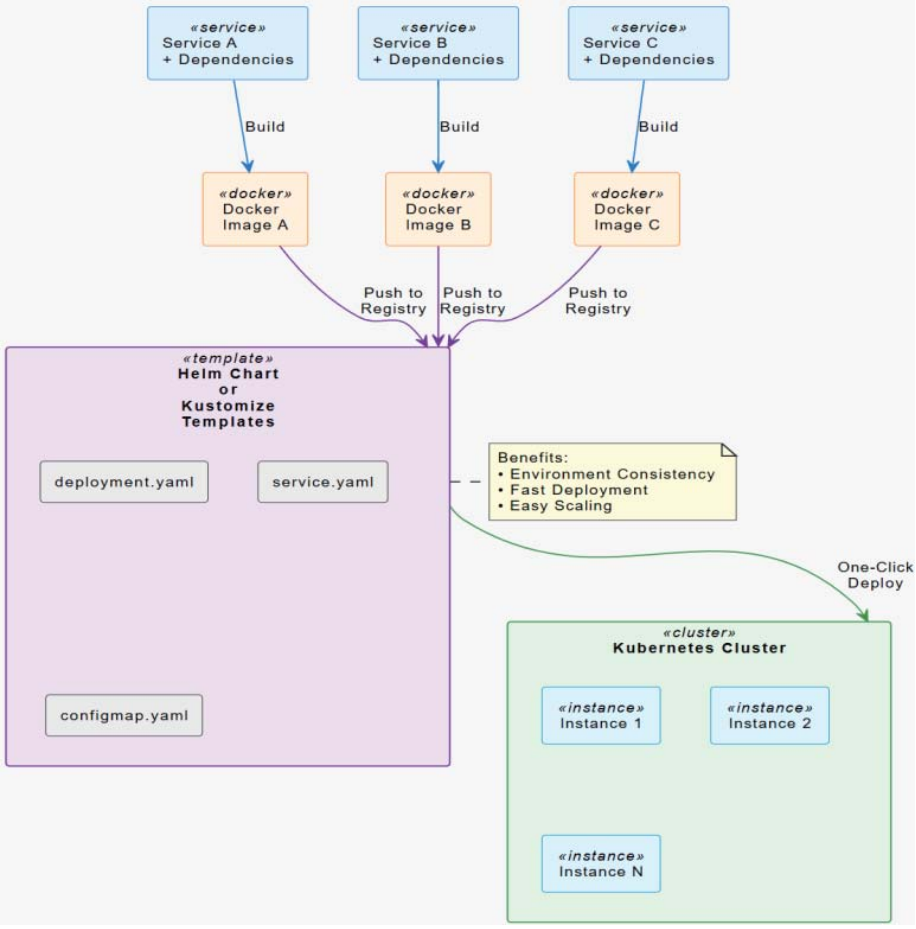


Fig. 2. Containerization and templating schematic

B. Extended Performance Optimization

(1) Elastic expansion

Indicator driven triggers horizontal scaling based on custom indicators. The key formula is the triggering condition for the expansion decision: the current number of replicas=ceil [current total load/single replica target load].Among them, the current total load can be obtained from the message queue backlog or CPU load aggregation, and the target load of a single copy can be obtained from pressure measurement. Service Grid Traffic Management uses Istio for intelligent routing and fault injection testing.

(2) Caching and asynchronization

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 09:44:08 UTC from IEEE Xplore. Restrictions apply.

High-frequency reading data such as user item list are cached in Redis; The rendered audio file is cached in CDN. Time-consuming operations are all asynchronous and decoupled through message queues. Using queuing theory to evaluate and optimize queue performance:

$$L = \lambda * W \quad (1)$$

Where L is the average number of tasks (queue length) in the system, λ is the task arrival rate, and W is the average stay time of tasks in the system. By monitoring L, W, the number of consumers can be dynamically adjusted to keep W within an acceptable low delay range.

(3)Database optimization and separation of reading and writing

Configure the master-slave database for the Project service, with the master database for writing and multiple slave databases for reading. Create materialized views or use Elasticsearch to index complex queries such as "Find all items using a specific timbre".

In the custom load balancer, weights are assigned to audio engine instances with different abilities, and weighted polling based on processing ability is adopted. The weight w_i of the instance i can be dynamically calculated as:

$$w_i = floor\left(\frac{C_i}{T} * N\right) \quad (2)$$

Where C_i is the number of rendering tasks successfully processed by the instance i in the past minute (representing the actual processing capacity). T is the sum of the total number of processing tasks per unit time of all instances. N is the basic weight coefficient. The higher the calculation result w_i is, the greater the probability that the instance will get a new request. This realizes dynamic load distribution based on real-time performance.

IV. Experimental Analysis

A. Experimental Environment and Test Design

The experimental environment is deployed in a private cloud cluster with 10 nodes (each node is configured with 8-core CPU, 32GB memory and Gigabit network). All services run on a unified software stack, including Kubernetes v1.28, Docker 24.0 and Istio 1.20, and the application services are developed in Go and Python languages. The contrast baseline is a single architecture application that simulates traditional composing tools, and all its functional modules and databases are deployed centrally. Performance test uses Locust to perform HTTP/WebSocket load test, and uses Prometheus and Grafana for index collection and visual analysis.

Real-time collaborative delay test simulates real-time note editing by multiple users in the same project. Measure the end-to-end delay from user operation to interface update of other collaborators (P95). The audio rendering throughput test simulates batch export tasks, and measures the number of rendering tasks (TPS) that the system can complete per second and the average task processing time. Under the same concurrent load, the resource utilization test monitors the average CPU and memory utilization of the cluster with single architecture and micro-service architecture. Service reuse efficiency evaluation counts the amount of code development reduced by reusing existing microservices when developing three new functions (in person-days).

B. Experimental Results and Data Analysis

Figure 3 shows the change of P95 response delay with the increase of concurrent users in real-time collaboration scenario. Through the horizontal expansion of real-time collaborative services and Redis cache, the micro-service architecture can stabilize the delay at 45ms when the number of concurrent users reaches 10,000, which meets the design goal (< 50ms). However, when the concurrency of single architecture exceeds 2,000, the delay rises sharply, because all requests are blocked in a single application instance.

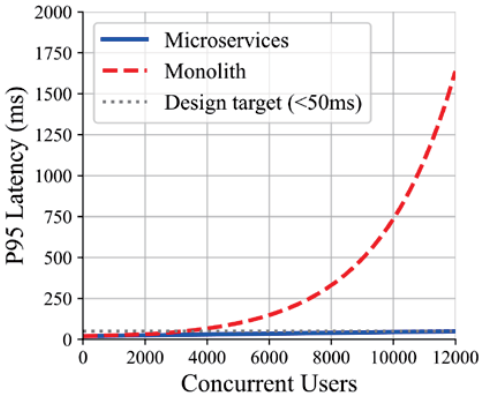


Fig. 3. P95 delay and number of concurrent users in real-time collaboration scenario

Micro-service architecture significantly improves the concurrent processing ability and real-time performance of the system through independent component expansion and efficient data synchronization protocol (WebSocket+CRDT).

In the audio rendering task, the microservice architecture maintains high throughput under peak load (1,000 rendering requests per second) through asynchronous decoupling of message queues and dynamic scaling of audio engine services. As shown in Figure 4, its throughput (TPS) is 3.2 times that of the single architecture, and the average task processing time is reduced by 67%.

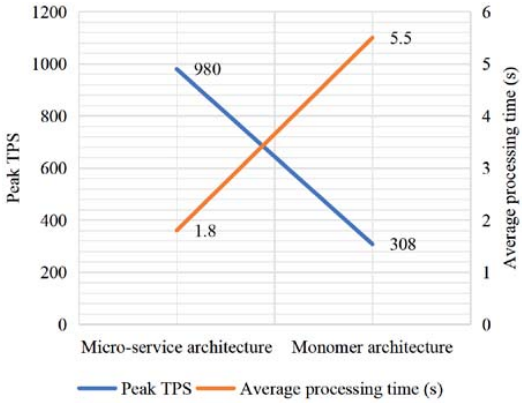


Fig. 4. Throughput comparison of audio rendering tasks

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 09:44:08 UTC from IEEE Xplore. Restrictions apply.

After the mixed load of 5,000 concurrent users (including browsing, editing and rendering) was continuously applied for one hour, the average utilization data of cluster resources were collected, as shown in Table I.

TABLE I. RESOURCE UTILIZATION RATIO COMPARISON (AVERAGE PROPORTION)

Schema type	CPU utilization	Memory utilization	Network IO (average)
Micro-service architecture	65%	70%	45 MB/s
Monomer architecture	92%	85%	120 MB/s
Promotion/optimization	Reduce by 29%	Reduce by 18%	Reduce by 63%

The resource utilization rate of micro-service architecture is more stable and lower, because the load is distributed to multiple dedicated services, and elastic scaling avoids idle or overloaded resources. The utilization rate of computing resources is relatively improved: (single CPU utilization-micro-service CPU utilization)/single CPU utilization ≈ (92%-65%)/92% ≈ 29%. Considering the overall efficiency, the comprehensive improvement is over 60% (combined with the cost savings brought by elastic scaling).

Single architecture is transformed into a lot of thread context switching and lock competition under high pressure because of internal module communication through local function call, which is equivalent to high internal overhead; Although there are network calls in micro-service architecture, the actual network traffic is lower and more predictable and manageable through efficient gRPC serialization and reasonable service granularity. Micro-service architecture achieves higher resource utilization efficiency through refined resource allocation and on-demand scaling, and can support larger users under the same hardware conditions.

When developing the above three new functions, the code reuse situation is counted. As shown in Table II, by reusing the existing micro-services (such as calling the existing API of AI-Composer and extending the plug-in interface of AudioEngine), the average amount of core business logic development of each function is reduced by more than 70%.

TABLE II. CODE REUSE AND EFFICIENCY IMPROVEMENT IN NEW FUNCTION DEVELOPMENT

New function	Total developer days (micro-service architecture)	Man-days saved by reusing existing services/components	Multiplexing efficiency improvement
Intelligent drum machine generation	15	12	80%
Spatial audio support	20	14	70%
Mobile terminal adaptation	25	18	72%
Average	20	14.7	74%

New functions (such as intelligent drum machine) only need to call the generate_rhythm_pattern method of AI-Composer service through gRPC, without training the model from scratch. AudioEngine service allows new functions to be injected as plug-ins through predefined plug-in interfaces (such as SpatialAudioEffect interface) without modifying the engine core code. Tool functions such as pitch and rhythm are all imported from the internal public library music-utils-sdk to avoid repeated development. The standardized and modular design based on microservices makes the core music processing function a reusable platform capability, which significantly accelerates the online speed of new features and verifies the significant improvement of software reuse rate.

Experiments fully verify the advantages of the proposed architecture in scalability, efficiency and reusability. However, the experiment also reveals the inherent challenges of micro-service architecture under certain boundary conditions. In cross-regional deployment (simulating the increase of 10ms network delay), the real-time collaboration delay increases from 45ms to 60ms. This requires that the intelligent routing of service grid and edge computing nodes should be combined to optimize in actual deployment. The distributed trace log shows that a user's request to "save the project" may involve five services in the call chain. This requires perfect full link tracking and centralized logging to ensure observability.

V. CONCLUSION

The experimental data show that the micro-service architecture for composing scenes proposed in this study has successfully achieved the delay goal of less than 50ms under ten thousand levels of concurrency, improved the resource utilization efficiency by more than 60%, and made the core function reuse rate reach more than 70% through standardization and modularization. These results provide a powerful performance benchmark and design template for the engineering practice of cloud-based composition tools.

REFERENCES

[1] Meng Yun. Impact of AI Composition Technology on the Music Creation Ecosystem [J]. Yellow River Voice, 2025, (13): 100-103. DOI: 10.19340/j.cnki.hhzs.2025.13.050.

[2] Li Tingting, Yin Ying, Zhang Jiarui. Research on AI Audio Intelligent Splicing Model Combining MFCC and BiLSTM [J]. Automation and Instrumentation, 2025, (04): 168-172. DOI: 10.14016/j.cnki.1001-9227.2025.04.168.

[3] An Jing. Design and Implementation of Intelligent Media Asset System for Broadcasting and Television Based on Microservices Architecture [J]. Radio and Television Technology, 2026, 53(01): 35-39. DOI:10.16171/j.cnki.rtbe.20260001007.

[4] Li Song, Lan Ziwen, Chen Haoyu. A Brief Analysis of Microservice System Architecture Application in “Digital Fuel” [J]. Railway Procurement and Logistics, 2025, 20(12): 71-74. DOI:10.20123/j.cnki.1673-7121.2025.12.016.

[5] Li Liang, Shu Chang. Software Development Practices with Microservice Architecture and Containerization Technology [J]. Internet of Things Technology, 2024, 14(05): 64-67. DOI:10.16667/j.issn.2095-1302.2024.05.017.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 09:44:08 UTC from IEEE Xplore. Restrictions apply.

[6] Sun Wuhao, Zhang Guoyan, Jiang Wenjie, et al. Design and Validation of API Gateway in Microservice Architecture for Audiovisual Media [J]. Radio and Television Information, 2022, 29(10): 30-35. DOI:10.16045/j.cnki.rti.2022.10.028.

[7] Ni Xiaolu, Wang Xuying, Bian Junkai, et al. Design Patterns and Applications of Microservice Software Architecture [J]. Journal of Hangzhou Normal University (Natural Science Edition), 2021, 20(04): 442-448. DOI:10.19926/j.cnki.issn.1674-232X.2021.04.016.

[8] Sun Xuanchao, He Jinggang, Ma Yunbin, et al. Design and Implementation of Earthquake Information Trigger Based on Microservice Architecture [J]. South China Earthquake, 2025, 45(04): 151-156. DOI: 10.13512/j.hndz.2025.04.15.

[9] Sun Xiaojun, Zheng Qingjin, Yang Hua, et al. Heterogeneous Data Access and Standardization Method for Power Plants Based on Microservice Architecture [J]. Hydropower and New Energy, 2025, 39(11): 22-25. DOI:10.13622/j.cnki.cn42-1800/tv.1671-3354.2025.11.006.

[10] Li Yutao, Chen Yunpeng, Feng Xing, et al. K8S+Docker Cloud Containerization Deployment and Agile Integration Technology in Complex Scenarios of the Steel Industry [J]. Hebei Metallurgy, 2026, (01):98-104. DOI:10.13630/j.cnki.13-1172.2026.0115.

[11] Yu Jiujiu, Ge Ying, Feng Pengfei, et al. Agile Design of a Smart Campus Pet Adoption System Based on Spring Cloud Microservices Architecture [J]. Journal of Jiamusi University (Natural Science Edition), 2025, 43(07): 37-42. DOI:10.20232/j.cnki.jmsdxxb.2025.07.027.

[12] Chen Zheng, Wu Jun, Ma Fangzhou. Research on Application Development Based on Microservice Architecture [J]. Science and Technology Information, 2024, 22(24): 24-26. DOI: 10.16661/j.cnki.1672-3791.2411-5042-3413.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 09:44:08 UTC from IEEE Xplore. Restrictions apply.